```
            loss = loss_fn(X, Y.detach())
            optimizer.zero_grad()
            loss.backward()
            losses.append(loss.item())
            optimizer.step()

        if reward != -1 or mov > max_moves:
15
            status = 0
            mov = 0
losses = np.array(losses)
```

- ***1*** Sets the total size of the experience replay memory
- ***2*** Sets the mini-batch size
- ***3*** Creates the memory replay as a deque list
- ***4*** Sets the maximum number of moves before game is over
- ***5*** Selects an action using the epsilon-greedy strategy
- ***6*** Computes Q values from the input state in order to select an action
- ***7*** Creates an experience of state, reward, action, and the next state as a tuple
- ***8*** Adds the experience to the experience replay list
- ***9*** If the replay list is at least as long as the mini-batch size, begins the mini-batch training
- ***10*** Randomly samples a subset of the replay list
- ***11*** Separates out the components of each experience into separate mini-batch tensors
- ***12*** Recomputes Q values for the mini-batch of states to get gradients
- ***13*** Computes Q values for the mini-batch of next states, but doesn't compute gradients
- ***14*** Computes the target Q values we want the DQN to learn
- ***15*** If the game is over, resets status and mov number

In order to store the agent's experiences, we used a data structure called a *deque* in Python's built-in collections library. It's basically a list that you can set a maximum size on, so that if you try to append to the list and it is already full, it will remove the first item in the list and add the new item to the end of the list. This means new experiences replace the oldest experiences. The experiences themselves are tuples of `(state1, reward, action, state2, done)` that we append to the `replay` deque.